

Java ArrayList

Basics → Master: Internals, Methods, Complexity & LeetCode Problem Bank

ArrayList isn't an algorithmic pattern like sliding window or two pointers — it's the tool those patterns run on top of. This guide covers what's actually happening under the hood (so you know WHY certain operations are fast/slow), the API you'll use constantly, the traps that cause silent bugs or runtime exceptions, and a problem bank where ArrayList itself (not just arrays) is the right data structure to reach for.

Section	Covers
1. What & Why	ArrayList vs array vs LinkedList, when to reach for it
2. Internal Mechanics	Dynamic array, capacity growth, amortized $O(1)$
3. Complexity Cheat Sheet	Time cost of every common operation
4. Core API & Templates	Methods, iteration, sorting, 2D lists, comparators
5. Problem Bank (Tiers 1-6)	24 curated problems, basics -> master
6. Fully Worked Examples	4 problems solved with full reasoning + code
7. Common Mistakes & Traps	ConcurrentModificationException, shallow copy, autoboxing
8. Practice Roadmap	Order to solve + spaced repetition schedule

1. What is ArrayList & Why It Exists

A Java array has a fixed size decided at creation — you cannot grow or shrink it. ArrayList is built on top of a plain array internally, but manages resizing for you automatically, giving you a dynamic, growable list while still keeping array-like $O(1)$ index access.

ArrayList vs Array vs LinkedList

- **Array (`int[]`, `String[]`):** fixed size, fastest possible, no autoboxing for primitives, use when size is known upfront.
- **ArrayList:** resizable, backed by an array internally, $O(1)$ random access, $O(n)$ worst-case insert/delete in the middle — use when size is unknown or you need list-style methods (add/remove/contains).
- **LinkedList:** $O(1)$ insert/delete at known nodes, $O(n)$ random access — use when you insert/remove at both ends frequently (also implements Deque) but rarely index into the middle.

Default choice rule: in interviews and on LeetCode, ArrayList (or plain array when size is fixed/known) is almost always the right default. Reach for LinkedList only when the problem specifically needs $O(1)$ insert/delete at arbitrary known positions or deque behavior.

When you'll actually use ArrayList in DSA problems

- Building an adjacency list for graphs: `List<List<Integer>>`
- Returning variable-length results (e.g. all subsets, all permutations, all paths) where size isn't known ahead of time.
- Backtracking, where you repeatedly add/remove the last element (ArrayList's add/remove-last is $O(1)$ amortized).
- Anywhere you'd use a Python list or JS array — ArrayList is Java's equivalent.

2. Internal Mechanics (the part most people skip)

Understanding this is what lets you reason about time complexity instead of guessing.

2.1 It's a dynamic array

- Internally, `ArrayList` holds a plain `Object[]` array called `elementData`.
- `size` (logical length, what `size()` returns) is tracked separately from `elementData.length` (physical capacity, which is usually bigger).
- `get(i)` and `set(i, val)` are direct array index operations — $O(1)$, same speed as a raw array.

2.2 The growth strategy — why `add()` is 'amortized' $O(1)$

- When you call `add()` and the internal array is full, `ArrayList` allocates a NEW array roughly 1.5x the old capacity, then copies every existing element over — this single call costs $O(n)$.
- But this expensive resize happens rarely (only when the array fills up), and doubling/1.5x growth means the total cost of n additions, summed over all resizes, is $O(n)$ — not $O(n^2)$. Divide that total by n operations and each `add()` costs $O(1)$ **on average** — this is what 'amortized $O(1)$ ' means.
- This is exactly why default-sized `ArrayLists` in a tight loop are still fast — the occasional expensive resize is 'hidden' by all the cheap $O(1)$ adds around it.

Practical tip: if you know the approximate final size upfront, use `new ArrayList<>(expectedSize)` to pre-allocate capacity and skip the resize copies entirely — this matters in performance-sensitive code, less so in typical LeetCode problems.

2.3 Why `remove(i)` / `add(i, val)` in the middle are $O(n)$

- Removing or inserting at index i requires shifting every element after i by one position — that's an $O(n)$ array copy (`System.arraycopy` under the hood), not a pointer update like `LinkedList`.
- Removing/adding at the END (index `size()-1` or `size()`) needs no shifting — that's the $O(1)$ amortized case.

3. Complexity Cheat Sheet

Operation	Time	Why
<code>get(i)</code> / <code>set(i, val)</code>	$O(1)$	Direct array index access.
<code>add(val)</code> (append at end)	$O(1)$ amortized	Occasional $O(n)$ resize, averaged over many calls.
<code>add(i, val)</code> (insert at index)	$O(n)$	Shifts all elements after i by one.
<code>remove(i)</code> (by index)	$O(n)$	Shifts all elements after i by one.
<code>remove(Object o)</code> (by value)	$O(n)$	Linear scan to find it, then $O(n)$ shift.
<code>contains(val)</code> / <code>indexOf(val)</code>	$O(n)$	Linear scan — no hashing like <code>HashSet</code> .
<code>size()</code> / <code>isEmpty()</code>	$O(1)$	size is tracked as a field, not computed.
Iterating (for-each)	$O(n)$	One pass, but each step is $O(1)$.
Sort (<code>Collections.sort</code>)	$O(n \log n)$	Uses TimSort (a merge sort variant).

Compare to `HashMap`: `contains()/get()` there is $O(1)$ average. If you find yourself calling `ArrayList.contains()` inside a loop, that's a strong signal you actually need a `HashSet` instead — a very common LeetCode optimization.

4. Core API & Templates

4.1 Essentials you should be able to write without looking up

```
List<Integer> list = new ArrayList<>();
list.add(5); // append
list.add(0, 10); // insert at index 0 -> O(n)
list.get(1); // read -> O(1)
list.set(1, 99); // overwrite -> O(1)
list.remove(0); // remove by INDEX -> O(n)
list.remove(Integer.valueOf(99)); // remove by VALUE -> O(n), note the boxing!
list.contains(5); // O(n) linear scan
list.size();
list.isEmpty();
Collections.sort(list); // ascending, O(n log n)
Collections.sort(list, Collections.reverseOrder()); // descending
```

4.2 Custom sort with a Comparator (very common in interviews)

```
List<int[]> intervals = new ArrayList<>();
// sort by start time ascending
intervals.sort((a, b) -> a[0] - b[0]);
// sort by end time, then start time as tiebreaker
intervals.sort((a, b) -> a[1] != b[1] ? a[1] - b[1] : a[0] - b[0]);
```

4.3 2D ArrayList (list of lists) — adjacency list pattern

```
int n = 5;
List<List<Integer>> graph = new ArrayList<>();
for (int i = 0; i < n; i++) graph.add(new ArrayList<>());

// add an edge u -> v
graph.get(u).add(v);

// iterate neighbors of node u
for (int neighbor : graph.get(u)) {
    // process neighbor
}
```

4.4 Safe removal while iterating (avoiding ConcurrentModificationException)

```
// WRONG - throws ConcurrentModificationException
for (Integer x : list) {
    if (x == 3) list.remove(x); // modifying list.size() mid for-each
}

// RIGHT - use an Iterator directly
Iterator<Integer> it = list.iterator();
while (it.hasNext()) {
    if (it.next() == 3) it.remove(); // safe, iterator tracks this
}

// ALSO RIGHT - removeIf (cleanest for simple conditions)
list.removeIf(x -> x == 3);
```

5. Problem Bank — Tier 1 to Tier 6 (24 problems)

These are problems where ArrayList (not a fixed array) is the natural or required data structure — variable-length output, list-of-lists, or backtracking with add/remove.

Tier 1 — Basic Manipulation

LC#	Problem	Diff.	Key Concept / Theory
1470	Shuffle the Array	Easy	Build result with a plain ArrayList/array using add() in a computed order.
1108	Defanging an IP Address	Easy	String + list manipulation basics, building output incrementally.
1929	Concatenation of Array	Easy	addAll() or manual double-loop append.
2011	Final Value of Variable After Performing Operations	Easy	Iterate a List of operations, mutate a running value.

Tier 2 — Variable-Length Output (why you need ArrayList, not array)

LC#	Problem	Diff.	Key Concept / Theory
78	Subsets	Med	Backtracking: add element, recurse, remove element (backtrack) — the canonical ArrayList add/remove-last pattern.
46	Permutations	Med	Same add/remove backtracking, with a 'used' boolean array to avoid reuse.
77	Combinations	Med	Backtracking with a start-index parameter to avoid duplicate combinations.
17	Letter Combinations of a Phone Number	Med	Backtracking building strings via a shared List result + StringBuilder.

Tier 3 — List of Lists (2D ArrayList patterns)

LC#	Problem	Diff.	Key Concept / Theory
797	All Paths From Source to Target	Med	DFS on a List> adjacency list, backtracking the current path list.
133	Clone Graph	Med	Build a new List/adjacency structure while doing DFS/BFS with a HashMap to avoid recloning.
994	Rotting Oranges	Med	BFS using a List (or ArrayDeque) as a queue of coordinate pairs.
102	Binary Tree Level Order Traversal	Med	Build List>, one inner ArrayList per tree level.

Tier 4 — Sorting & Custom Comparators

LC#	Problem	Diff.	Key Concept / Theory
56	Merge Intervals	Med	Sort List by start time with a Comparator, then merge overlapping ones by mutating the ArrayList in place.

LC#	Problem	Diff.	Key Concept / Theory
253	Meeting Rooms II	Med	Sort start times and end times separately (two ArrayLists/arrays), sweep with two pointers.
973	K Closest Points to Origin	Med	Sort a List with a distance-based Comparator, or use a heap for large n.
452	Minimum Number of Arrows to Burst Balloons	Med	Sort List by end coordinate — greedy + Comparator combo.

Tier 5 — Mutating While Iterating / In-Place List Tricks

LC#	Problem	Diff.	Key Concept / Theory
1089	Duplicate Zeros	Easy	In-place ArrayList/array shifting — careful with the $O(n)$ shift-by-one-index cost.
287	Find the Duplicate Number	Med	Treat array values as a linked structure (fast-slow), showing ArrayList/array can simulate other structures.
41	First Missing Positive	Hard	In-place index-as-hash-key trick using array/ArrayList swaps to avoid extra space.

Tier 6 — Master Tier (combined with other structures)

LC#	Problem	Diff.	Key Concept / Theory
order	Design a data structure with getRandom() $O(1)$	Med	LC 380 (Insert Delete GetRandom $O(1)$): ArrayList + HashMap combo — swap-with-last-element trick to keep remove() $O(1)$.
order2	LRU Cache-style list usage	Med	LC 146 variant using ArrayList vs LinkedHashMap — compare why LinkedList/HashMap combo beats plain ArrayList for $O(1)$ LRU eviction.
23	Merge k Sorted Lists	Hard	Collect all values into an ArrayList, sort, rebuild — contrast this $O(n \log n)$ approach against the optimal heap-based $O(n \log k)$.
134	Gas Station	Med	Greedy simulation over an ArrayList/array with a single pass — good integration problem to close out this tier.

6. Fully Worked Examples (with reasoning)

Example A — 78. Subsets (the ArrayList backtracking template)

Problem: Return all possible subsets of a distinct-integer array.

Reasoning

- Use ONE mutable ArrayList (the 'current path') that you add() to before recursing, and remove the last element from (backtrack) after recursing.
- At every recursive call, the current state of that list IS a valid subset — record a COPY of it (new ArrayList<>(current)), never the reference itself (it keeps mutating).
- This add-recurse-remove shape is the single most reusable ArrayList pattern in all of backtracking.

```
public List<List<Integer>> subsets(int[] nums) {
    List<List<Integer>> result = new ArrayList<>();
    backtrack(nums, 0, new ArrayList<>(), result);
    return result;
}

private void backtrack(int[] nums, int start, List<Integer> current, List<List<Integer>> result) {
    result.add(new ArrayList<>(current)); // snapshot copy, not the live reference

    for (int i = start; i < nums.length; i++) {
        current.add(nums[i]);           // choose
        backtrack(nums, i + 1, current, result); // explore
        current.remove(current.size() - 1); // un-choose (backtrack), O(1) since it's the last element
    }
}
```

Example B — 56. Merge Intervals (sort + in-place ArrayList mutation)

Problem: Merge all overlapping intervals.

Reasoning

- Sort the intervals by start time first — this guarantees that once you've processed interval i, any future overlap can only come from intervals that start before the current merged interval ends.
- Keep a result ArrayList. For each interval: if it overlaps with the LAST interval already in result, merge by extending that last interval's end; otherwise, add it as a new entry.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) -> a[0] - b[0]); // sort by start time
    List<int[]> merged = new ArrayList<>();

    for (int[] interval : intervals) {
        if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < interval[0]) {
            merged.add(interval); // no overlap with last -> new entry
        } else {
            merged.get(merged.size() - 1)[1] =
                Math.max(merged.get(merged.size() - 1)[1], interval[1]); // overlap -> extend end
        }
    }
    return merged.toArray(new int[merged.size()][]);
}
```

Example C — 380. Insert Delete GetRandom O(1) (ArrayList + HashMap combo)

Problem: Design a structure supporting insert, remove, and getRandom, all in average $O(1)$.

Reasoning — why plain `ArrayList.remove(value)` doesn't work

- `getRandom()` needs $O(1)$ index-based access, which only an `ArrayList` (or array) gives you — a `HashSet` has no index.
- But removing by `VALUE` from an `ArrayList` is normally $O(n)$ (needs to find it, then shift). The trick: instead of shifting, SWAP the element to remove with the `LAST` element, then `removeLast` ($O(1)$).
- A `HashMap` tracks where every value currently lives in the `ArrayList`, so you can find-and-swap in $O(1)$ instead of scanning.

```
class RandomizedSet {
    private List<Integer> list = new ArrayList<>();
    private Map<Integer, Integer> valueToIndex = new HashMap<>();
    private Random rand = new Random();

    public boolean insert(int val) {
        if (valueToIndex.containsKey(val)) return false;
        valueToIndex.put(val, list.size());
        list.add(val);
        return true;
    }

    public boolean remove(int val) {
        if (!valueToIndex.containsKey(val)) return false;
        int indexToRemove = valueToIndex.get(val);
        int lastValue = list.get(list.size() - 1);

        list.set(indexToRemove, lastValue);          // move last element into the gap
        valueToIndex.put(lastValue, indexToRemove);  // update its index in the map
        list.remove(list.size() - 1);                //  $O(1)$  removal from the end
        valueToIndex.remove(val);
        return true;
    }

    public int getRandom() {
        return list.get(rand.nextInt(list.size())); //  $O(1)$  index access
    }
}
```

7. Common Mistakes & Traps

- **ConcurrentModificationException:** modifying a list's structure (add/remove) inside a for-each loop over it throws this. Use an explicit Iterator + `it.remove()`, or `removeIf()`, instead.
- **remove(int) vs remove(Object):** `list.remove(5)` removes the element AT INDEX 5. `list.remove(Integer.valueOf(5))` removes the VALUE 5. Mixing these up on a `List<Integer>` is a classic silent bug — the code compiles but does the wrong thing.
- **Autoboxing overhead:** `List<Integer>` stores boxed Integer objects, not primitive ints — every add/get involves boxing/unboxing. For huge numeric workloads this matters (`int[]` is faster); for typical LeetCode sizes it doesn't.
- **Shallow copy trap:** `new ArrayList<>(otherList)` copies the list structure, but if elements are objects/arrays, both lists still reference the SAME inner objects — mutating one affects the other. Watch for this in backtracking (Example A) and matrix problems.
- **subList() returns a VIEW, not a copy:** `list.subList(0, 3)` is backed by the original list — modifying it can throw `ConcurrentModificationException` or silently mutate the original. Wrap in `new ArrayList<>(list.subList(0,3))` if you need an independent copy.
- **== vs equals() on Integer:** Integer caches values -128 to 127, so `Integer.valueOf(100) == Integer.valueOf(100)` is true but `Integer.valueOf(200) == Integer.valueOf(200)` is false. Always use `.equals()` (or auto-unbox to compare as int) — never `==` on boxed types.
- **toArray() type traps:** `list.toArray()` returns `Object[]`, not the specific type — use `list.toArray(new Integer[0])` or a manual loop when you need a typed array back.

8. Practice Roadmap & Spaced Repetition

Suggested order (2-3 problems/day)

- **Days 1-2:** Tier 1 (all 4 problems) — comfort with basic add/get/set/remove.
- **Days 3-6:** Tier 2 (all 4 problems) — the add-recurse-remove backtracking template; this is the highest-leverage skill in this whole guide.
- **Days 7-10:** Tier 3 (all 4 problems) — List<List<Integer>> as adjacency lists and level-order results.
- **Days 11-14:** Tier 4 (all 4 problems) — sorting with custom Comparators, a skill needed constantly across all patterns, not just ArrayList problems.
- **Days 15-17:** Tier 5 (all 3 problems) — in-place index tricks.
- **Days 18-21:** Tier 6 — attempt cold, these combine ArrayList with HashMap/heap for $O(1)$ or $O(\log n)$ operations.

Spaced repetition after first solve

- Day 1: solve with full struggle (aim 25-30 min before hints).
- Day 3: re-solve cold — for Tier 2 problems specifically, check you can write the add/recurse/remove skeleton from memory without pausing.
- Day 7: re-solve cold again; note which trap from Section 7 you almost fell into.
- Day 21: final cold re-solve — fast, error-free completion means the ArrayList mechanics are now second nature.

Same method as your Sliding Window and Two Pointers guides — attempt first, use this only after genuine struggle.